

INTRODUCTION AU LANGAGE SQL

SQL (Structured Query Language) a été introduit par IBM comme le langage d'interface de son prototype de système de gestion de base de donnée relationnelle, System-R. Le premier système SQL disponible sur le marché a été introduit en 1979 par Oracle.

Aujourd'hui, SQL est devenu un standard de l'industrie et Oracle est un leader dans la technologie des systèmes de gestion de bases de données relationnelles.

Comme SQL est un langage non procédural, des ensembles d'enregistrements peuvent être manipulés à la fois. La syntaxe est naturelle et souple, ce qui permet de se concentrer sur la présentation des données.

SQL–le standard

L'ANSI (American National Standards Institute) a déclaré SQL le langage standard pour les systèmes de gestion de bases de données relationnelles. La plupart des entreprises qui produisent des systèmes de gestion de bases de données relationnelles sont compatibles avec SQL et essaient de respecter le standard SQL89.

Les facettes de SQL

Le langage SQL possède trois facettes:

- ❖ Langage de définition de données (LDD)
(Création et évolution du schéma relationnel).
- ❖ Langage de manipulation de données (LMD)
(Interrogation et options de mises à jour).
- ❖ Langage de contrôle de données (LCD)
(gestion de l'intégrité et confidentialité).

CHAPITRE 1 : DEFINITION DE DONNEES DANS LE LANGAGE SQL

1- Définitions

La définition de données dans SQL permet la description des objets manipulés par le SGBD. Les commandes du LDD sont :

- **CREATE** : création des objets.
- **ALTER** : modification de structure de l'objet.
- **DROP** : Suppression d'objets.

2- Gestion des tables

2-1- Création de la table

Celle-ci consiste à définir son *nom*, les *colonnes* qui la composent et leurs *types*. Elle se fait avec la commande : **CREATE TABLE**

Syntaxe :

CREATE TABLE nom_table
(col1 type [(taille)] [default Valeur Par Defaut] [null/not null] [contrainte de colonne],
col2 type [(taille)] [default Valeur Par Defaut] [null/not null] [contrainte de colonne],
coln type [(taille)] [default Valeur Par Defaut] [null/not null] [contrainte de colonne],
[Contrainte de tables]) ;

a- Définition des colonnes :

- La **taille** indique la valeur maximale de la longueur du champ.
- Les **types** de données possibles sont :
 - **NUMBER** : stocke les nombres en général entiers positifs et négatifs.
NUMBER(n) ou NUMBER(n,m)
n : varie de 1 à 38 et indique le nombre maximum des chiffres à stocker.
m : indique le nombre de chiffres situés à droite de la virgule.
 - **FLOAT** : stocke des nombres en virgule flottante.
 - **CHAR** : stocke des chaînes comportant au maximum 255 caractères dans une colonne de longueur fixe : CHAR(n). Si on ne spécifie pas la longueur : CHAR, une colonne de ce type ne stocke qu'un seul caractère.
 - **VARCHAR / VARCHAR2** : le deuxième type remplace le premier. Il stocke jusqu'à 4000 caractères dans une seule colonne pour des chaînes de longueur variable.

- **DATE** : permet de stocker des informations concernant les dates et les heures.

EX :

Créer une table correspondante à l'entité client avec comme clé le champ numéroclient. Cette table comprend les champs relatifs au nom, téléphone, adresse, code postal et ville.

```
CREATE TABLE CLIENT  
( numclt number(6) not null,  
  nom varchar (15),  
  tel number(8),  
  adr varchar(20),  
  cod_pos number(4),  
  ville char(12));
```

b- Définition des contraintes :

Une contrainte d'intégrité est une règle qui définit la cohérence d'une donnée ou d'un ensemble de données de la base de données.

SQL définit certaines contraintes telles que :

- **Non nullité des valeurs d'un attribut** : L'utilisateur doit saisir la valeur de l'attribut. La commande est : **NOT NULL**

- **Unicité de la valeur d'un attribut ou d'un groupe d'attributs** : la valeur doit être unique. La commande est : **UNIQUE**.

- **Clé primaire (un attribut ou un groupe)** : indique que l'attribut est une clé primaire. Elle peut être définie comme contrainte de table ou comme contrainte de colonne.

❖ Clé primaire comme contrainte de table selon la syntaxe :

CONSTRAINT nom_contrainte PRIMARY KEY (att1, att2,...,attn) ;

❖ Clé primaire comme contrainte de colonne : en ajoutant devant la colonne clé primaire **Primary key**.

Remarque :

Dans le cas de clé primaire multiple, la clé primaire doit être comme contrainte de table (Syntaxe 1).

Exemple 1 : Clé primaire comme contrainte de table

```
create table piece (codpiece number(5), despiece varchar(10), coulpiece varchar(10),  
constraint pk_piece primary key (codpiece)) ;
```

Exemple 2 : Clé primaire comme contrainte de colonne

```
create table piece (codpiece number(5) primary key, despiece varchar(10), coulpiece  
varchar(10));
```

Exemple 3 : Clé primaire multiple

Create table LigneCde (Num_cde number(5), NumProd number(5), Datecde date, Qtecde number(5), constraint pk_lignecde primary key (Num_cde, NumProd));

- **Clé étrangère (intégrité référentielle)** : Lorsque la clé primaire figure dans une autre table en tant qu'un attribut non clé.

La clé étrangère peut être définie comme contrainte de table ou comme contrainte de colonne.

- ❖ Clé étrangère comme contrainte de table selon la syntaxe :

CONSTRAINT nom_contrainte FOREIGN KEY (nom_att) references nom_table (nom_att) ;

- ❖ Clé étrangère comme contrainte de colonne : en ajoutant devant la colonne clé étrangère **references nom_table (nom_att)**.

Remarque :

Il est impossible de créer une clé étrangère si la clé primaire associée n'existe pas.

Exemple 1 : Clé étrangère comme contrainte de table

Create table vente
(codvnt number(6) primary key,
article number(5),
qte number(5),
constraint fk_vente foreign key (article) references produit (codprod));

Exemple 2 : Clé étrangère comme contrainte de colonne

Create table vente
(codvnt number(6) primary key,
article number(5) references produit (codprod),
qte number(5));

- **Contrainte de valeur avec la clause check** : Permet de limiter les valeurs possibles pour une colonne en vérifiant une certaine condition. Le contrôle se fera lors des insertions des données.

CONSTRAINT nom_contrainte check (colonne condition)

La condition sur la colonne peut utiliser :

- ❖ Un opérateur de comparaison.
- ❖ La clause between val1 and val2.
- ❖ La clause in (liste de valeurs).

Exemples :

1-

```
create table commande  
(codvnt number(6) primary key,  
article number(5),  
qte number(5),  
constraint ck_commande check (qte >0));
```

2-

```
create table produit  
(codprod number(5) primary key,  
desprod varchar(10),  
coulprod varchar(10),  
constraint ck_produit check (coulprod in ('r','g','b')));
```

Application:

Créer la base de données suivante:

Magasin (NumMag : numérique de taille 8, surface : numérique de taille 5, ville : caractère variable de taille 15)

Produit (NumProd : numérique de taille 7, Couleur : caractère variable de taille 10, Qte : numérique sur 5 positions, Designation : caractère variable de taille 10, Poids : numérique sur 7 positions dont 3 après la virgule, #NumMag)

Commande (NumCde : numérique de taille 5, DateCde : date)

LigneCde (#NumCde, #NumProd, QteCde : numérique de taille 9)

En considérant les hypothèses suivantes :

- ❖ La ville est par défaut Tunis
- ❖ La couleur ne peut être que Rouge, Bleu, Vert ou Jaune.
- ❖ La quantité en stock doit être >0
- ❖ La quantité en stock doit être saisie
- ❖ Le poids doit être compris entre 0 et 100.

Correction :

```
create table Magasin  
(NumMag number(8) primary key,  
surface number (5),  
ville varchar(15) default 'Tunis');
```

```
create table Produit  
(NumProd number(7) primary key,  
Couleur varchar(10),  
Qte number(5) not null,  
Designation varchar(10),
```

```
Poids number (7,3),
NumMag number(8) references Magasin (NumMag),
constraint ck1_pro check (Poids>0),
constraint ck2_pro check (Couleur in ('Rouge','Bleu','Vert','Jaune')),
constraint ck3_pro check (Poids between 0 and 100));
```

```
create table Commande
(NumCde number(5) primary key,
DateCde date));
```

```
create table LigneCde
(NumCde number(5),
NumProd number (7),
QteCde number(9),
constraint pk_ligne primary key (NumCde,NumProd),
constraint fk1_ligne foreign key (NumCde) references Commande (NumCde),
constraint fk2_ligne foreign key (NumProd) references Produit (NumProd));
```

2-2- Modification des structures des tables

ORACLE définit trois possibilités de modification de structure de table qui permettent soit d'ajouter des colonnes, soit de modifier la structure d'une colonne, soit de supprimer des colonnes existantes.

1^{ère} forme : Ajout de nouvelles colonnes à une table

Syntaxe :

```
ALTER TABLE nom_table
ADD (col1 type [(taille)] [null / not null],
    col2 type [(taille)] [null / not null],
    ...
    coln type [(taille)] [null / not null]);
```

Rq:

Quand on veut ajouter un champ en lui choisissant son emplacement:

```
ADD coln type [(taille)] [null / not null] before colp
```

Ex:

Supposons qu'on veut ajouter une colonne type_pdt à la table produit:

```
ALTER TABLE produit
ADD (type_pdt char(3)) ;
```

2^{ème} forme : Modification de la structure d'une colonne existante

Syntaxe :

```
ALTER TABLE nom_table
MODIFY (col1 type [(taille)] [null / not null],
        col2 type [(taille)] [null / not null],
        ...
        coln type [(taille)] [null / not null]);
```

Ex :

```
ALTER TABLE produit
MODIFY (type_pdt char (5)) ;
```

Rq:

Pour modifier le nom d'une colonne :

```
RENAME COLUMN nom_table.ancien_nom TO nom_table.nouveau_nom ;
```

3^{ème} forme : Suppression de colonnes existantes

Syntaxe :

```
ALTER TABLE nom_table
DROP (col1, col2,.....,coln) ;
```

Ex :

Supposons qu'on veut supprimer le champ num_fax de la table fournisseur :

```
ALTER TABLE fournisseur DROP num_fax ;
```

4^{ème} forme : Ajout d'une contrainte

Syntaxe :

```
ALTER TABLE nom_table
ADD Constraint Def_de_contrainte;
```

Ex:

1- Ajout de contrainte clé primaire:

```
ALTER TABLE emp ADD constraint clep PRIMARY KEY (empno);
```

2- Ajout de contrainte clé étrangère:

```
ALTER TABLE emp ADD constraint cletran FOREIGN KEY (Mgr) references dept
(deptno);
```

5^{ème} forme : Suppression de contraintes existantes

a- Suppression d'une contrainte clé primaire :

On peut effacer une clé primaire. La commande est :

ALTER TABLE nom_table DROP PRIMARY KEY [CASCADE];

Rq:

L'option cascade est ajoutée pour pouvoir supprimer une clé primaire référencée.

b- Suppression d'une contrainte autre que la clé primaire :

On peut effacer une clé étrangère. La commande est :

ALTER TABLE nom_table DROP CONSTRAINT nom_contrainte ;

Où le nom de la contrainte c'est celui de la contrainte à supprimer.

Remarque :

Pour retrouver les différentes contraintes avec leurs propriétés, on peut utiliser la commande suivante :

**Select * from user_constraints
[where table_name = 'NOMTABLE'];**

Il est à remarquer que pour cette commande, le nom de la table doit être écrit en majuscule.

2-3- Suppression de tables

Syntaxe :

DROP TABLE nom_table ;

Ex :

Supposons qu'on veut supprimer la table client_tunis :

DROP TABLE client_tunis ;

2-4- Renommage et création de synonymes de tables

Pour changer le nom d'une table existante, la commande est :

Syntaxe :

RENAME ancien_nom TO nouveau_nom ;

Il est également possible de donner à une même table plusieurs noms différents appelés synonymes.

Syntaxe :

CREATE SYNONYM nom_synonyme FOR nom_table ;

Pour supprimer un synonyme donné, on utilise la commande :

Syntaxe :

DROP SYNONYM nom_synonyme ;

Rq :

La suppression d'une table implique la suppression des synonymes correspondants.

CHAPITRE 2 : MANIPULATION DE DONNEES DANS LE LANGAGE SQL

1- Insertion de données

a- 1^{ère} forme : Insertion de valeurs pour la totalité des valeurs de la table

Syntaxe :

INSERT INTO nom_table [(les champs de la table)] VALUES (liste des valeurs) ;

Les valeurs des colonnes de type caractère ou chaîne de caractères doivent apparaître entre deux cotes (''). Si la valeur est nulle on doit mettre deux cotes successives (sans blanc).

Ex :

INSERT INTO produit VALUES ('P007','chaise',98.12, 15) ;

EX :

On peut ne pas mettre certaines colonnes de la table s'il en existe une valeur qui sera remplie par défaut, par exemple le champ ville est par défaut SFAX :

INSERT INTO Magasin (NumMag, surface) VALUES (203,150) ;

b- 2^{ème} forme : Insertion à travers la copie des valeurs des colonnes d'une autre table

Syntaxe :

INSERT INTO nom_table [(les champs de la table)] Requête ;

Ex :

INSERT INTO client (num_clt, nom_clt, tel_clt) select * from client_bizerte;

2- Modification de données

L'opération de modification consiste à mettre à jour les colonnes d'une table par des nouvelles valeurs.

Syntaxe :

**UPDATE nom_table
SET col1 = val1,..., coln = valn
WHERE condition ;**

Rq :

- Il n'est pas possible de mettre à jour plus qu'une table à la fois.
- Les valeurs peuvent être des constantes, des expressions ou des résultats de sous requêtes.

Ex :

Modifier l'adresse du client TRIKI qui devient 15, cité des fleurs.

UPDATE client SET adr_clt = ' 15, cité des fleurs' WHERE nom_clt = 'TRIKI' ;

3- Suppression de données

Syntaxe :

**DELETE FROM nom_table
WHERE condition;**

Ex:

- 1) Supprimer toutes les commandes qui datent avant le 28/04/99.

DELETE FROM commande WHERE date_cde < '28/04/1999';

- 2) Supprimer les commandes passées par le client TRIKI.

DELETE FROM commande WHERE num_clt = (select num_clt from client where nom = 'TRIKI');

4- Consultation de données

4-1- Présentation

SQL est basé sur l'utilisation de blocks de qualification dont la structure est composée principalement de trois éléments :

- ❖ SELECT : permet de définir la liste des colonnes que l'on peut obtenir.
- ❖ FROM : n'indique que les noms des tables nécessaires pour obtenir le résultat souhaité.
- ❖ WHERE : définit la condition que doit vérifier un n-uplet donné pour qu'il fasse partie du résultat.

Syntaxe :

**SELECT col1, col2,..., coln
FROM nom_table
[WHERE condition];**

Si on souhaite toutes les colonnes de la table, on mettra après SELECT *.

Si on souhaite toutes les lignes de la table, on ne met pas la clause WHERE.

a- Notion d'Alias

Chaque fois qu'une requête d'interrogation est exécutée, les noms des colonnes définies dans la structure de la table apparaissent comme titres de colonnes. On peut modifier ces noms de colonnes, à l'affichage uniquement, en ajoutant des alias.

La commande devient :

```
SELECT col1 [alias 1], col2 [alias 2],..., coln [alias n]  
FROM nom_table  
[WHERE condition];
```

b- Condition de sélection

La condition est composée généralement de trois termes :

- ❖ Un nom de colonne,
- ❖ Un opérateur de comparaison
- ❖ Et une constante ou une colonne ou une liste de valeurs.

En plus des opérateurs classiques de comparaison (=, <>, <, >, >=, <=) ORACLE dispose d'un certain nombre d'opérateurs spécifiques dont principalement :

- **IS NULL** : permet de tester si le contenu d'une colonne est une valeur nulle (indéfinie).
- **IN** (liste de valeurs) : permet de tester si le contenu d'une colonne coïncide avec l'une des valeurs de la liste.
- **BETWEEN V and V2** : permet de tester si le contenu d'une colonne est compris entre les valeurs V1 et V2.
- **LIKE** chaîne générique : permet de tester si le contenu d'une colonne ressemble à une chaîne de caractères obtenues à partir de la chaîne générique. La chaîne générique est une chaîne de caractères qui contient l'un des caractères suivants :
 - % : remplace une autre chaîne de caractères qui peut être même une chaîne vide.
 - : remplace un seul caractère.

Tous les opérateurs spécifiques peuvent être mis sous forme négative en les faisant précéder de l'opérateur de négation **NOT** : **NOT IN, NOT BETWEEN, NOT LIKE, IS NOT NULL**.

EX :

1) Lister tous les produits dont la quantité en stock est inférieur à 20.
SELECT * FROM produit WHERE qtestk <20;

2) Sélectionner la désignation et le prix unitaire des produits dont le prix unitaire est compris entre 5 et 10.
SELECT design, prixU FROM produit WHERE prixU BETWEEN 5 and 10;

3) Sélectionner tous les clients de Sfax et de Tunis.
SELECT * FROM client WHERE ville IN ('Sfax', 'Tunis');

4) Sélectionner la désignation et le prix unitaire des produits disponibles dont le prix unitaire est compris entre 5 et 10.

SELECT design, prixU FROM produit WHERE qtestk>0 and prixU BETWEEN 5 and 10;

5) Sélectionner les noms des clients qui commencent par la lettre T.
SELECT nom FROM client WHERE nom LIKE 'T%';

6) Sélectionner les noms des clients dont la deuxième la lettre est R.
SELECT nom FROM client WHERE nom LIKE '-R%';

7) Attribuer au produit n° 3 le même prix unitaire que le produit ayant le n°1.
UPDATE produit SET prisU = (SELECT prixU FROM produit WHERE num_pdt = 1)
WHERE num_pdt = 3;

8) Majorer de 5% les prix unitaires des produits dont le prix est supérieur à 10.
UPDATE produit SET prisU = prixU * 1.05 WHERE prixU > 10 ;

4-2- Les fonctions s'appliquant aux chaînes de caractères

- **RTRIM (ch)** : supprime l'espace à la fin de la chaîne.
- **RPAD (ch, n)** : ajoute n espaces à la fin de la chaîne.
- **INTCAP (ch)** : met en majuscule la première lettre de chaque mot de la chaîne.
- **SUBSTR (ch, m, n)** : permet d'extraire la sous-chaîne de ch qui commence à partir du caractère à la position m et de longueur n.
- **TRANSLATE (ch, ch1, ch2)** : permet de transformer dans la chaîne ch toutes les occurrences de ch1 par ch2.

Ex :

```
Magasins (magnum, magloc, magger)
M1      sfax   triki
M2      tunis  fki
```

```
SELECT SUBSTR (magger, 2, 3) FROM Magasins;
Donne: rik
      Ki
```

4-3- Les expressions s'appliquant à des dates

- **ADD_MONTHS (d, n)** : permet d'ajouter n mois à la date d sachant que n est un entier.
- **GREATEST (d1, d2)** : permet d'avoir la date le plus récente parmi d1 et d2.
- **MONTHS_BETWEEN (d1, d2)** : permet d'avoir le nombre de mois qui se trouvent entre la date d1 et la date d2.
- **LAST_DAY (d)** : permet d'avoir la date du dernier jour de la date d.
- **SYSDATE** : donne la date et l'heure système.

Ex :

LAST_DAY ('02/01/02') donne : 31/01/02

4-4- Les fonctions de conversion

- **TO_CHAR (valeur-date, format-date) / TO_CHAR (nombre, [format])** : convertit une date ou une valeur numérique à une chaîne de caractères.
- **TO_DATE (valeur-chaîne, format-date)** : convertit une chaîne de caractères représentant une date à une date.
- **TO_NUMBER (ch, [format])**: convertit une chaîne de caractères représentant un nombre en nombre.

Rq :

Quelques formats de fonctions :

YYYY	Année sans virgule
YYY	3 derniers chiffres de l'année
YY	2 derniers chiffres de l'année
Y	Dernier chiffre de l'année
MM	Numéro du mois (1 à 12)
DD	Numéro de jour dans le mois (1 à 31)
D	Numéro de jour dans la semaine (1 à 7)
SS	Secondes (1 à 60)
YEAR	Année en toute lettre
MON	Nom du mois abrégé en toute lettre
DAY	Nom du jour sur 9 caractères
SP	Nombre en toute lettre

Ex :

Ventes (numvnt, ..., datvnt,...)
 1 03/01/00

SELECT To_CHAR(datvnt,'YEAR') FROM ventes;
Donne: two thousands

4-5 Les expressions agrégats (ou fonctions de groupe)

ORACLE dispose d'un ensemble de fonctions appelées fonctions agrégats qui s'appliquent à un ensemble de données :

- **AVG** : permet d'avoir la moyenne arithmétique d'un ensemble donné.
- **COUNT** : permet d'avoir le nombre d'occurrences des enregistrements.
- **MAX** : permet d'avoir la valeur maximale dans une colonne.
- **MIN** : permet d'avoir la valeur minimale dans une colonne.
- **SUM** : permet d'avoir la somme des éléments.
- **STDDEV** : permet d'avoir l'écart type.
- **VARIANCE** : permet d'avoir la variance.

Chacune de ces fonctions a comme argument un nom de colonne ou une expression arithmétique. Elles ignorent les valeurs nulles et par défaut prennent les valeurs multiples pour des valeurs différentes. Pour ne prendre que les valeurs distinctes, il faut ajouter l'opérateur **DISTINCT**.

La fonction COUNT peut prendre comme argument le caractère * ce qui veut dire que nous voulons connaître le nombre de lignes sélectionnées.

Rq :

On ne peut pas mettre une fonction de groupe après la clause WHERE parce qu'elle s'agit d'une valeur inconnue.

Ex :

Sélectionner tous les étudiants qui ont une note supérieure à la note moyenne de tous les étudiants.

```
SELECT * FROM etudiant WHERE note_etu > (SELECT AVG (note_etu) FROM etudiant);
```

EX :

- 1) Sélectionner les noms distincts des clients qui ont fait des commandes.

```
SELECT DISTINCT nom_clt from commande;
```

- 2) Trouver le nombre de clients de la ville de Sfax

```
SELECT COUNT (*) from client where ville = 'Sfax';
```

4-6- Classification des résultats

Il est possible de regrouper (classifier) les résultats obtenus permettant d'avoir ensemble les lignes ayant une même valeur pour la colonne de classification. La classification permet aussi d'effectuer un calcul tel que la moyenne ou la somme des valeurs de chaque groupe et de présenter le résultat sous forme d'une ligne par groupe.

La clause qui permet la classification est :

GROUP BY expression

[HAVING condition]

Ex:

Trouver la somme des quantités commandées par produit et par client qui sont supérieures à 10.

```
SELECT num_pdt, num_clt, SUM (qte_cde) FROM commande  
GROUP BY num_pdt, num_clt  
HAVING SUM(qte_cde) >10;
```

4-7- Tri des résultats

Pour obtenir un résultat trié, il suffit d'ajouter à la requête SQL la clause :

ORDER BY expression [asc / desc]

Rq:

Si on ne spécifie pas asc ou desc par défaut le tri est croissant (asc).

Ex :

Donner la liste des produits par ordre de prix unitaire croissant et ceux ayant le même prix par ordre alphabétique décroissant.

```
SELECT * FROM produit ORDER BY prixU, design desc;
```

4-8- Les requêtes imbriquées

La requête imbriquée figure lorsqu'une sous requête apparaît dans la clause WHERE de la requête principale.

Ex :

1) Trouver la désignation des produits dont le prix unitaire est égal à celui des chaises.
`SELECT design FROM produit WHERE prixU = (SELECT prixU FROM produit WHERE design = 'chaise');`

2) Trouver les produits dont le prix est supérieur au prix moyen.
`SELECT design FROM produit WHERE prixU > (SELECT AVG (prixU) FROM produit);`

4-9- Les opérateurs ensemblistes

Requête 1

Opérateur

Requête 2

Avec l'utilisation de ces opérateurs :

- Il faut utiliser le même nombre de variables avec les mêmes types dans les deux requêtes.
- Les doublons sont automatiquement éliminés.
- Les titres des colonnes résultats sont ceux de la première requête.

a- L'union : **UNION**

Elle permet d'avoir les résultats de la première requête suivis de ceux de la deuxième requête.

Ex :

Donner les noms des clients dont la matricule est C001 ou C002 ou C003 et les noms des clients qui habitent Sfax.

```
SELECT nom FROM client WHERE num_clt IN ('C001','C002','C003')  
UNION
```

```
SELECT nom FROM client WHERE ville = 'Sfax';
```

b- L'intersection : **INTERSECT**

Elle permet d'avoir les lignes communes aux résultats des deux requêtes.

Ex :

Donner les noms des clients dont la matricule est C001 ou C002 ou C003 et en même temps qui habitent Sfax.

```
SELECT nom FROM client WHERE num_clt IN ('C001','C002','C003')
```

INTERSECT

```
SELECT nom FROM client WHERE ville = 'Sfax';
```

c- La différence : MINUS

MINUS permet d'avoir les lignes qui apparaissent dans la première requête et qui n'apparaissent pas dans la seconde.

Ex :

Donner les noms de tous les clients sauf ceux qui habitent Sfax.

```
SELECT nom FROM client
```

```
MINUS
```

```
SELECT nom FROM client WHERE ville = 'Sfax';
```

4-10- Produit cartésien et jointure

a- Produit cartésien

Le produit cartésien consiste à croiser toutes les données d'une table avec celles d'une autre table. Pour construire les n-uplets résultants, on combine chaque n-uplet de la première table avec tous les n-uplets de la seconde table.

Avec SQL, on spécifie les tables à croiser dans la clause FROM et les colonnes résultantes dans la clause SELECT.

Ex :

Lister toutes les commandes (num_cde, dat_cde, num_pdt, num_clt,qte_cde) avec pour chaque commande la date de la vente correspondante.

```
SELECT C. num_cde, C. dat_cde, C. num_pdt, C. num_clt, C. qte_cde, V.dat_vnt FROM  
commande C, vente V;
```

b- La jointure

- **La jointure simple**

Elle permet de faire un rapprochement de deux tables par comparaison d'une ou de plusieurs colonnes communes aux deux tables. La jointure est un sous ensemble du produit cartésien.

Les colonnes utilisées pour la comparaison doivent être de même type et de même taille et sont appelées colonnes de jointure.

La forme générale d'une requête de jointure est :

```
SELECT col1, col2,..., coln
```

```
FROM table1,...,tablen
```

```
WHERE condition;
```

Ex:

1) Trouver le nom du client qui a passé la commande n° 124.

```
SELECT nom FROM client CL, commande CM
```

```
WHERE CL.idclt = CM.idclt AND numcde = 124;
```

2) Donner une liste des commandes avec en plus le nom du client.

```
SELECT CM.*, nom FROM client CL , commande CM  
WHERE CL.idclt = CM.idclt;
```

- **L'autojointure**

Elle consiste à faire le rapprochement d'une table avec elle-même ; c'est-à-dire ramener sur une même ligne des informations qui proviennent de plusieurs lignes de la même table. Pour cela il faut créer un SYNONYM de la table ou utiliser des variables ALIAS.

Ex :

Trouver la désignation et le prix unitaire des produits dont le prix est supérieur à celui du produit chaises.

1^{ère} solution:

```
CREATE SYNONYM prod FOR produit ;  
SELECT prod.design, prod.prixU FROM prod, produit WHERE produit.design = 'chaise'  
AND prod.prixU > produit.prixU;
```

2^{ème} solution:

```
SELECT p1.design, p1.prixU FROM produit p1, produit p2  
WHERE p2.design = 'chaise' AND p1.prixU > p2.prixU;
```

5- Manipulation de données à travers les vues

La notion de vue permet de manipuler les données autrement qu'à travers les tables définies dans la base. Une vue est stockée sous forme de requête de sélection dans le dictionnaire de données et permet de constituer les schémas externes.

L'utilisation des vues permet de :

- Restreindre l'accès à certaines colonnes et certaines lignes d'une table en autorisant un ou plusieurs utilisateurs à manipuler cette table qu'à travers une vue.
- Simplifier la tâche de l'utilisateur en le déchargeant de la formulation de requêtes complexes.

5-1- Création d'une vue

Elle consiste à définir sa structure c'est-à-dire son nom, les colonnes qui la composent et les données qui lui sont rattachées.

Syntaxe :

```
CREATE VIEW nom_de_la_vue [(col1,..., coln)]  
AS requête;
```

Rq :

Si on supprime la table de laquelle est issue la vue, la vue sera inutilisable.

Ex :

```
CREATE VIEW vprod (des, qte, prix)
AS SELECT design, SUM (qte_stk), AVG (prixU * qtestk)
FROM produit
GROUP BY design;
```

5-2- Suppression d'une vue

Elle consiste de supprimer du dictionnaire de données la définition correspondante à cette vue. Les données rattachées à la vue restent dans leurs tables.

Syntaxe :

DROP VIEW nom_de_la_vue ;

5-3- Consultation d'une vue

Toutes les requêtes de consultation possibles sur les tables le sont aussi sur les vues.

5-4- Mise à jour d'une vue

- Si la vue dérive de plusieurs tables ou si sa définition contient la clause GROUP BY, DISTINCT ou n'importe quelle fonction de groupe , alors les opérations **DELETE**, **INSERT** et **UPDATE** sont **interdites**.
- Si un attribut déclaré NOT NULL dans la table de base n'est pas repris dans la vue alors **aucune insertion n'est autorisée sur cette vue**.

TRAVAUX DIRIGES

TD N°1

LE LANGAGE SQL

Base de données exemple

Considérons la base de données dont le schéma est donné ci-dessous.

EMP (EMPNO, ENAME, JOB, #MGR, HIREDATE, SAL, COMM, #DEPTNO)

DEPT (DEPTNO, DNAME, LOC)

Clés primaires

#Clés étrangères

Supposons que la table EMP est déjà créée et remplie, elle a la structure suivante :

EMP

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17/12/80	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	20/02/81	1600.00	300.00	30
7521	WARD	SALESMAN	7698	22/02/81	1250.00	500.00	30
7566	JONES	MANAGER	7839	02/04/81	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	28/09/81	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	01/05/81	2850.00	NULL	30
7782	CLARK	MANAGER	7839	09/06/81	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	09/11/81	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	17/11/81	5000.00	NULL	10
7844	TURNER	SALESMAN	7698	08/09/81	1500.00	0.00	30
7876	ADAMS	CLERK	7788	23/09/81	1100.00	NULL	20
7900	JAMES	CLERK	7698	03/12/81	950.00	NULL	30
7902	FORD	ANALYST	7566	03/12/81	3000.00	NULL	20
7934	MILLER	CLERK	7782	23/01/82	1300.00	NULL	10

La table DEPT aura après création et remplissage la structure suivante :

DEPT

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW-YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Travail demandé :

Création de la base de données, contraintes d'intégrité

1) Créer la table DEPT. Ne pas oublier de définir le numéro de département DEPTNO comme clé primaire. Intégrer également la *contrainte de domaine* suivante : le nom d'un département (DNAME) ne peut être que ACCOUNTING, RESEARCH, SALES ou OPERATIONS.

2) Remplir la table DEPT.

- 3) Ajouter les contraintes d'intégrité nécessaires à la table EMP (clé primaire et clés étrangères) à l'aide de la commande ALTER TABLE EMP ADD CONSTRAINT...
- 4) Dans EMP, parmi ces nouveaux employés quelles insertions on peut effectuer, justifier :

(7369, 'WILSON', 'MANAGER', 7839, '17/11/91', 3500.00, 600.00, 10);
(7657, 'WILSON', 'MANAGER', 7839, '17/11/91', 3500.00, 600.00, 50);
(7657, 'WILSON', 'MANAGER', 7000, '17/11/91', 3500.00, 600.00, 10);
(7657, 'WILSON', 'MANAGER', 7839, '17/11/91', 3500.00, 600.00, 10);

Mise à jour de la base de données

- 1) Changer la localisation (LOC) du département SALES de CHICAGO à PITTSBURGH.
- 2) Dans EMP, augmenter de 10 % le salaire (SAL) des vendeurs dont la commission (COMM) est supérieure à 50 % du salaire.
- 3) Dans EMP, attribuer aux employés en poste avant le 01/01/82 (HIREDATE) et ayant une commission non spécifiée (NULL) une commission égale à la moyenne des commissions.
- 4) Dans DEPT, supprimer le département n° 20 (DEPTNO).

Interrogation de la base de données

Exprimer en SQL*Plus les requêtes suivantes.

- 1) Nom (ENAME), salaire, commission, salaire+commission de tous les vendeurs (SALESMAN).
- 2) Nom des vendeurs dont la commission est inférieure à 25% de leur salaire.
- 3) Nombre d'employés du département n° 10.
- 4) Nombre d'employés ayant une commission.
- 5) Nombre de fonctions (JOB) différentes.
- 6) Nom des employés gagnant plus que JONES.
- 7) Nom des employés occupant la même fonction que JONES.
- 8) Nom des employés ayant même manager (MGR) que CLARK.

CORRECTION DU TD N°1 LE LANGAGE SQL

Création de la base de données, contraintes d'intégrité

```
1) CREATE TABLE DEPT (DEPTNO NUMBER (2),
DNAME CHAR (20),
LOC CHAR (20),
CONSTRAINT D_CLEP PRIMARY KEY (DEPTNO),
CONSTRAINT D_DOM CHECK (DNAME IN
('ACCOUNTING','RESEARCH','SALES','OPERATIONS')));

2) INSERT INTO DEPT VALUES (10, 'ACCOUNTING', 'NEW-YORK');
INSERT INTO DEPT VALUES (20, 'RESEARCH', 'DALLAS');
INSERT INTO DEPT VALUES (30, 'SALES', 'CHICAGO');
INSERT INTO DEPT VALUES (40, 'OPERATIONS', 'BOSTON');

3) ALTER TABLE EMP ADD CONSTRAINT E_CLEP PRIMARY KEY (EMPNO);
ALTER TABLE EMP ADD CONSTRAINT E_CLET1 FOREIGN KEY (DEPTNO)
REFERENCES DEPT (DEPTNO);
ALTER TABLE EMP ADD CONSTRAINT E_CLET2 FOREIGN KEY (MGR)
REFERENCES EMP (EMPNO);
4) INSERT INTO EMP VALUES
(7657, 'WILSON', 'MANAGER', 7839, '17/11/91', 3500.00, 600.00, 10);
```

Mise à jour de la base de données

```
1) UPDATE DEPT SET LOC='PITTSBURGH' WHERE DNAME='SALES';
2) UPDATE EMP SET SAL=SAL*1.1 WHERE COMM>0.5*SAL;

3) UPDATE EMP SET COMM = (SELECT AVG (COMM) FROM EMP)
WHERE HIREDATE<'01/01/82' AND COMM IS NULL;

4) DELETE FROM DEPT WHERE DEPTNO=20;
```

Interrogation de la base de données

```
1) SELECT ENAME, SAL, COMM, SAL+COMM FROM EMP WHERE
JOB='SALESMAN';

2) SELECT ENAME FROM EMP WHERE COMM<.25*SAL;

3) SELECT COUNT (EMPNO) FROM EMP WHERE DEPTNO=10;
4) SELECT COUNT (EMPNO) FROM EMP WHERE COMM IS NOT NULL;
5) SELECT COUNT (DISTINCT JOB) FROM EMP;

6) SELECT ENAME FROM EMP WHERE SAL > (SELECT SAL FROM EMP WHERE
ENAME='JONES');
```

7) SELECT ENAME FROM EMP WHERE JOB = (SELECT JOB FROM EMP WHERE ENAME='JONES') AND ENAME<>'JONES';
8) SELECT ENAME FROM EMP WHERE MGR = (SELECT MGR FROM EMP WHERE ENAME='CLARK') AND ENAME<>'CLARK';

TD N°2 LE LANGAGE SQL
--

Exercice 1 :

Soit le schéma relationnel de la base FABRICATION.

CLIENT (NOC, NOM, ADRESSE)

SERVICE (NOS, INTITULE, LOCALISATION)

PIECE (NOP, DESIGNATION, COULEUR, POIDS)

COMMANDE (NOP, NOS, NOC, QUANTITE)

Clés primaires

Clés étrangères

Formuler en langage SQL les commandes de création de la structure de cette base, puis exprimer les requêtes suivantes :

- 1) Donner pour chaque service le poids de la pièce commandée de couleur bleue la plus pesante.
- 2) Donner le poids moyen des pièces commandées pour chacun des services "Promotion".
- 3) Donner les pièces de couleur bleue qui sont commandées par plus de trois services différents.
- 4) Donner le maximum parmi les totaux des quantités des pièces commandées par les différents services.

Exercice 2 :

Soit le schéma relationnel de la base de données « pilotes-avions-vols » :

PILOTE (PLNUM, PLNOM, PLPRENOM, VILLE, SALAIRE)

AVION (AVNUM, AVNOM, CAPACITE, LOCALISATION)

VOL (VOLNUM, PLNUM, AVNUM, VILLEDEP, VILLEARR, HEUREDEP, HEUREARR)

Clés primaires

Clés étrangères

Exprimer les requêtes suivantes en langage SQL :

- 1) Liste de tous les vols.
- 2) Nom, prénom et ville de tous les pilotes, par ordre alphabétique.
- 3) Nom, prénom et salaire des pilotes dont le salaire est supérieur à 20 000 F.
- 4) Numéro et nom des avions localisés à Paris.
- 5) Caractéristiques (AVNUM, AVNOM, CAPACITE, LOCALISATION) des avions localisés dans la même ville que le pilote Tanguy.

- 6) Caractéristiques (VOLNUM, VILLEDEP, VILLEARR, HEUREDEP, HEUREARR, AVNOM, PLNOM) du vol numéro 714.
- 7) Nom, prénom et numéro de vol des pilotes affectés à un vol.
- 8) Numéro et nom des avions affectés à des vols.
- 9) Nombre total de vols.
- 10) Somme des capacités par type (nom) d'avion.
- 11) Moyenne des durées des voyages.

CORRECTION DU TD N°2 LE LANGAGE SQL

Exercice 1 :

```
CREATE TABLE CLIENT (NOC NUMBER (3),  
NOM VARCHAR (40),  
ADRESSE VARCHAR (100),  
CONSTRAINT PRICLI PRIMARY KEY (NOC));
```

```
CREATE TABLE SERVICE (NOS NUMBER (3),  
INTITULE VARCHAR (30),  
LOCALISATION VARCHAR (100),  
CONSTRAINT PRISER PRIMARY KEY (NOS));
```

```
CREATE TABLE PIECE (NOP NUMBER (3),  
DESIGNATION VARCHAR (30),  
COULEUR VARCHAR (20),  
POIDS NUMBER (5, 2),  
CONSTRAINT PRIPIE PRIMARY KEY (NOP));
```

```
CREATE TABLE COMMANDE (NOP NUMBER (3),  
NOS NUMBER (3),  
NOC NUMBER (3),  
QUANTITE NUMBER (3),  
CONSTRAINT PRICOM PRIMARY KEY (NOP, NOS, NOC),  
CONSTRAINT ETRPIE FOREIGN KEY (NOP)  
REFERENCES PIECE (NOP),  
CONSTRAINT ETRSER FOREIGN KEY (NOS)  
REFERENCES SERVICE (NOS)  
CONSTRAINT ETRCLI FOREIGN KEY (NOC)  
REFERENCES CLIENT (NOC));
```

```
1) SELECT INTITULE, MAX (POIDS)  
FROM SERVICE S, COMMANDE C, PRODUIT P  
WHERE S.NOS=C.NOS  
AND C.NOP=P.NOP  
AND COULEUR='bleu'  
GROUP BY INTITULE;
```

```
2) SELECT AVG (POIDS)  
FROM SERVICE S, COMMANDE C, PRODUIT P  
WHERE S.NOS=C.NOS  
AND C.NOP=P.NOP  
AND INTITULE='Promotion'  
GROUP BY S.NOS;
```

```
3) SELECT P.NOP
FROM PRODUIT P
WHERE COULEUR='bleu'
AND 3 <
(SELECT COUNT (DISTINCT NOS)
FROM COMMANDE C
WHERE C.NOP=P.NOP);
```

```
4) SELECT MAX(SUM(QUANTITE))
FROM COMMANDE
GROUP BY NOS;
```

Exercice 2:

```
1) select * from vol;
2) select plnom, plprenom, ville from pilote order by plnom, plprenom;
3) select plnom, plprenom, salaire from pilote where salaire>20000;
4) select avnum, avnom from avion where localisation='Paris';
5) select avnum, avnom, capacite, localisation from avion a, pilote p
where a.localisation=p.ville and plnom='Tanguy';
6) select volnum, villedep, villearr, heuredep, heurearr, avnom, plnom
from avion a, pilote p, vol v where p.plnum=v.plnum and a.avnum=v.avnum and
volnum=714;
7) select plnom, plprenom, avnum from pilote p, vol v where v.plnum=p.plnum;
8) select distinct a.avnum, avnom from avion a, vol v where a.avnum=v.avnum;
9) select count(*) from vol;
10) select avnom, sum(capacite) from avion group by avnom;
11) select avg(heurearr-heuredep) from vol;
```